

FULL-STACK WEB APPLICATION



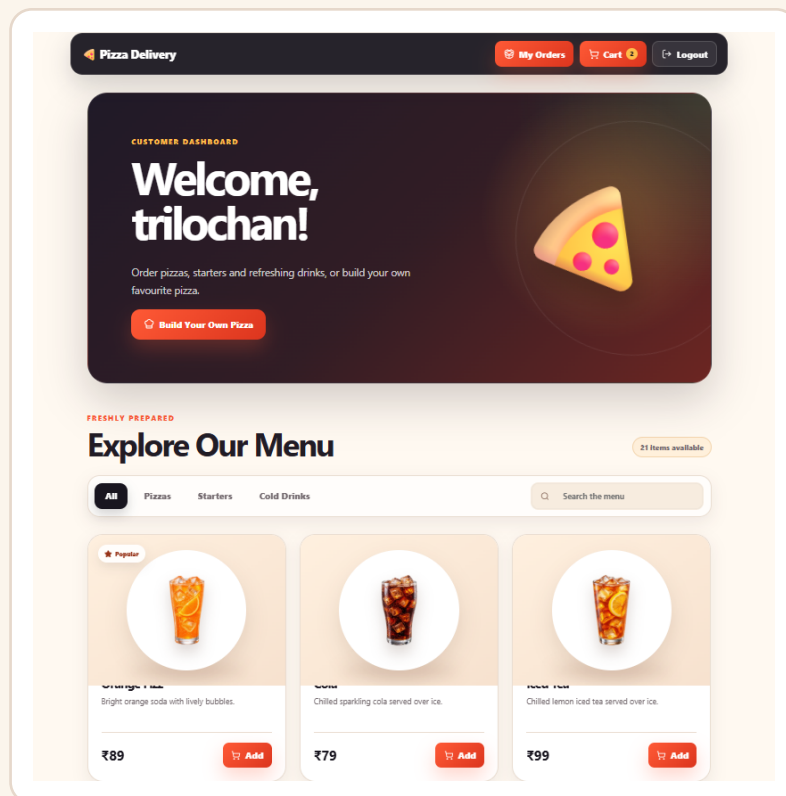
Pizza Delivery Platform

Custom ordering, payments, order tracking and administration

PROJECT REPORT

Oasis Infobyte Web Development Internship

Level 3 - Pizza Delivery Application | August 2026



Siba Sundar Sahoo

B.Tech - Computer Science and Engineering
ITER, Siksha 'O' Anusandhan

DEPLOYED FULL-STACK PROJECT

React | Node.js | MongoDB Atlas

01 / EXECUTIVE SUMMARY

A practical food-ordering system, built end to end

The Pizza Delivery Platform is a deployed full-stack application that supports the complete journey from account creation to order fulfilment. Customers can browse an expanded catalogue, build a custom pizza, manage a cart, choose a payment method and follow an order. Administrators can monitor customers, inventory and order progress from a separate protected workspace.

21

menu products

2

user roles

2

payment options

6

order states

Project snapshot

Area	Implementation
Customer application	Responsive React interface with catalogue search, category filters, custom pizza builder, cart, checkout and order history.
Backend	Express API with Mongoose models, JWT authentication, email flows, payment verification and administrator routes.
Cloud deployment	Frontend on Vercel, API on Render and application data in MongoDB Atlas.
Visual identity	A custom Ember & Basil theme using charcoal navigation, warm cream surfaces, coral actions and circular food imagery.

What makes the project complete

The work is not limited to a static menu. Authentication, email verification, persisted orders, payment handling, inventory visibility, deployment configuration and production testing are connected into one usable system.



02 / CONTEXT AND OBJECTIVES

Turning a familiar ordering flow into a reliable system

A pizza-ordering interface looks simple from the outside, but a complete implementation must keep product data, custom selections, customer identity, payment state and fulfilment status consistent across several screens and services.

Problem statement

Customers need a clear way to explore products, personalise an order and understand the final price before placing it. Restaurant staff need a different view: they must see inventory, customer demand and the state of each order without accessing customer-only pages. The project addresses both sides through role-based interfaces backed by one API and database.

Primary objectives

- Build secure registration, verification, login and password-recovery flows.
- Create a searchable catalogue covering pizzas, starters and cold drinks.
- Support custom pizzas without losing the selected ingredient details after checkout.
- Calculate subtotals, delivery fees and final totals consistently.
- Offer Cash on Delivery and Razorpay test-mode payments.
- Persist orders and present a clear five-stage delivery timeline to customers.
- Provide administrators with inventory and order-management tools.
- Deploy the application publicly and verify the production environment.

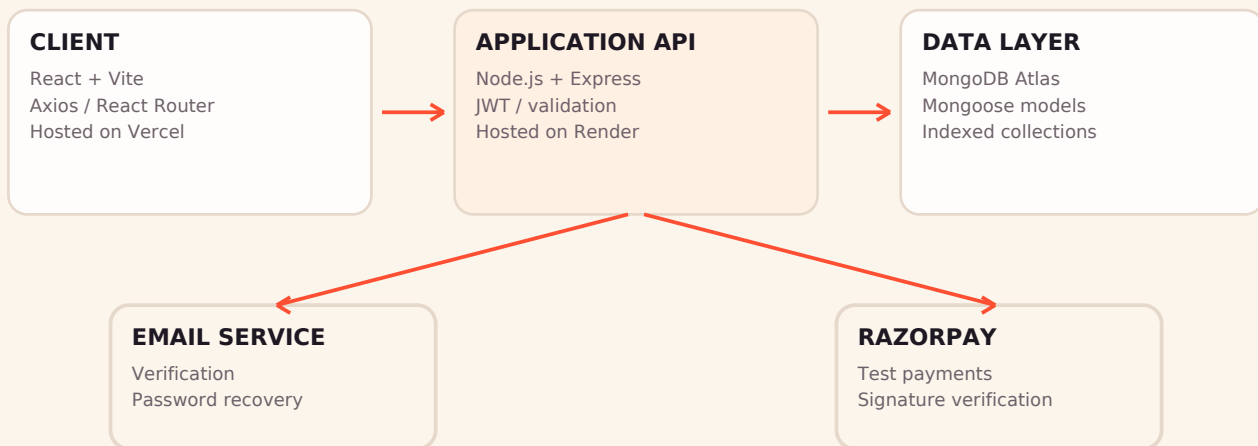
Scope decision

The implementation prioritises a dependable single-restaurant workflow. Advanced logistics such as live rider GPS and multi-branch routing are left for future versions so that the core ordering experience remains coherent and testable.

03 / SYSTEM ARCHITECTURE

A separated client, API and data layer

The application follows a three-tier structure. The React client focuses on interaction and presentation, the Express service owns business rules and security, and MongoDB stores durable application data. External services are isolated behind backend routes so sensitive credentials never need to be exposed in the browser.



Layer	Responsibility	Why it matters
React client	Pages, navigation, form state, catalogue browsing and user feedback.	Keeps the interface responsive and separates presentation from data rules.
Express API	Authentication, validation, catalogue queries, order creation, payment verification and admin operations.	Provides one controlled entry point to application data.
MongoDB Atlas	Users, ingredients, products, orders and status data.	Supports nested order snapshots and flexible catalogue records.
External services	Email delivery and Razorpay test payments.	Adds verification and payment capability without placing secrets in the frontend.

04 / USER JOURNEYS

Two roles, connected by the same order lifecycle

The customer and administrator experiences are intentionally separate. Each role receives only the controls needed for its work, while both views read the same underlying order record.

Customer journey	Administrator journey
1. Register and verify email 2. Log in securely 3. Browse or search 21 products 4. Build a custom pizza if required 5. Review quantities in Cart 6. Enter delivery details 7. Select COD or Razorpay 8. Follow progress in My Orders	1. Log in with an admin account 2. Review customer and order totals 3. Inspect ingredient inventory 4. Find pending customer orders 5. Update fulfilment status 6. Monitor low-stock indicators 7. Confirm operational changes from one dashboard

Order lifecycle



Shared source of truth

When an administrator changes an order from placed to confirmed, preparing, out for delivery or delivered, the customer timeline reflects the same status. This avoids maintaining separate copies of operational state.

05 / TECHNOLOGY DECISIONS

Tools chosen for speed, clarity and deployability

The stack was selected to keep the project understandable while still covering the concerns of a production-style application: authentication, validation, external services, persistent data and separate deployments.

Technology	Use in the project	Reason for selection
React + Vite	Customer and admin interfaces	Component-based development with fast local builds and a compact deployment output.
React Router	Protected and public navigation	Keeps login, customer and administrator routes explicit.
Axios	Client-to-API requests	Centralises API calls and authentication headers.
Node.js + Express	REST application server	Matches the frontend language and supports a modular middleware structure.
MongoDB + Mongoose	Application persistence	Works well with products, ingredients and nested order-item snapshots.
JWT + bcrypt.js	Authentication and password security	Supports stateless API access while storing only password hashes.
Nodemailer	Verification and recovery email	Provides controlled transactional email from the backend.
Razorpay	Online payment test flow	Adds a realistic Indian payment workflow and server-side signature verification.
Vercel + Render	Cloud hosting	Separates static frontend delivery from the long-running API service.

Using JavaScript across the client and server reduced context switching during implementation. MongoDB also made it practical to preserve a snapshot of ordered items, including custom pizza selections, rather than relying only on product references that might later change.



06 / DATA AND BACKEND DESIGN

Records designed around the order, not only the menu

The data model stores operational facts at the point they become important. Product records describe the current catalogue; order records preserve what the customer actually purchased, the delivery destination and the state of fulfilment.

Model	Important fields	Purpose
User	name, email, password hash, role, verification state	Controls identity, access and account lifecycle.
Ingredient	name, category, price, stock and availability	Feeds the custom pizza builder and admin inventory view.
Pizza / Menu Item	name, description, price, image, menu section and featured state	Represents pizzas, starters and cold drinks in one catalogue.
Order	order number, user, item snapshots, address, totals, payment and status fields	Preserves the complete commercial and fulfilment record.

Why order snapshots matter

An order should not change when a menu item is renamed or repriced later. For that reason, each order item stores its own name, image, price and quantity. A custom pizza can also retain its selected base, sauce, cheese and vegetables. The order therefore remains readable in My Orders even when catalogue data evolves.

Consistency checks

- An order must contain at least one item.
- Monetary values are constrained to non-negative numbers.
- Payment and order status values use defined enumerations.
- Order lookups are indexed by customer and creation time.
- Razorpay identifiers are stored separately from the public order number.

07 / AUTHENTICATION AND SECURITY

Protecting accounts, routes and deployment secrets

Security was treated as a cross-cutting concern rather than a single login screen. Passwords are hashed, protected endpoints require access tokens, account email is verified and sensitive deployment values remain outside the repository.

Control	Implementation
Password storage	bcrypt.js hashing; plain-text passwords are never stored.
Session access	JWT access and refresh tokens with protected customer and admin routes.
Email ownership	Verification links, resend support and password-recovery messages through the backend mail service.
Role separation	Customer and administrator pages are guarded independently.
Request protection	Helmet, CORS rules, rate limiting and server-side validation.
Payment integrity	Razorpay payment details are verified on the server before marking an online payment as complete.
Secret management	MongoDB, mail, JWT and payment secrets are stored in local or platform environment variables and excluded from Git.

Public documentation, private configuration

The README shows only placeholder environment variables. Real connection strings, app passwords and payment secrets remain in backend/.env locally and in protected Render settings for production.

Authentication flow

A new customer registers, receives a verification message and confirms the address before normal use. Login returns application tokens, and protected API calls include the access token. If the password is forgotten, a time-limited recovery flow allows the customer to choose a new one without revealing the old password.

08 / CUSTOMER CATALOGUE

A dashboard designed around quick discovery

The customer dashboard combines a strong entry point with practical catalogue controls. The user can switch between all products, pizzas, starters and cold drinks, or search directly by product name.

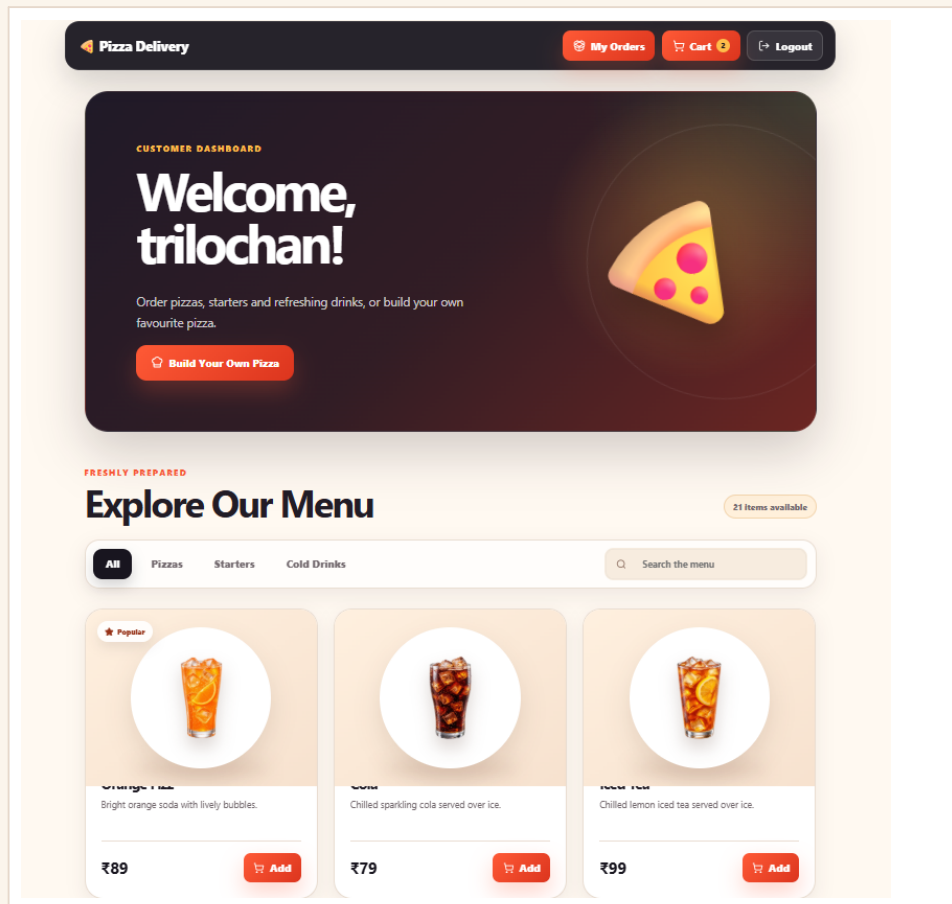


Figure 1. Customer dashboard with category filters, search and circular menu imagery.

10

pizzas

6

starters

5

cold drinks

21

total products

09 / CUSTOM PIZZA BUILDER

Personalisation without losing price clarity

The builder divides a potentially complicated selection process into four steps: base, sauce, cheese and vegetables. A live summary keeps the chosen components visible and allows the customer to understand the resulting pizza before it enters the cart.

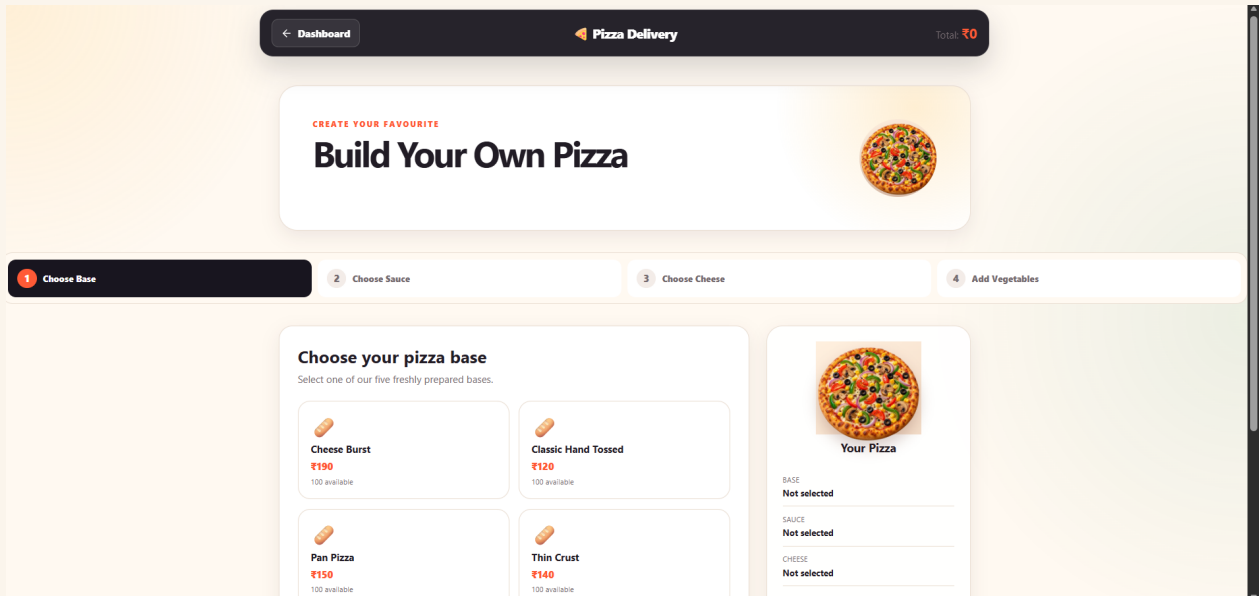


Figure 2. Step-based custom pizza builder with ingredient availability and live selection summary.

Design decisions

Decision	Reason
Four visible steps	Reduces cognitive load and makes progress predictable.
Availability counts	Connects the customer choice to restaurant inventory.
Live pizza summary	Prevents accidental submission of incomplete selections.
Stored custom snapshot	Keeps the final configuration visible in Cart, Checkout and My Orders.

This feature required coordination across the catalogue response, frontend selection state, cart representation and order model. Treating a custom pizza as a first-class order item prevented the builder from becoming an isolated demo screen.

10 / CART AND CHECKOUT

Clear totals before the order becomes permanent

The Cart focuses on review: product identity, quantity, unit price and removal controls remain visible beside a concise order summary. Checkout then separates delivery information from payment choice while keeping the same totals in view.

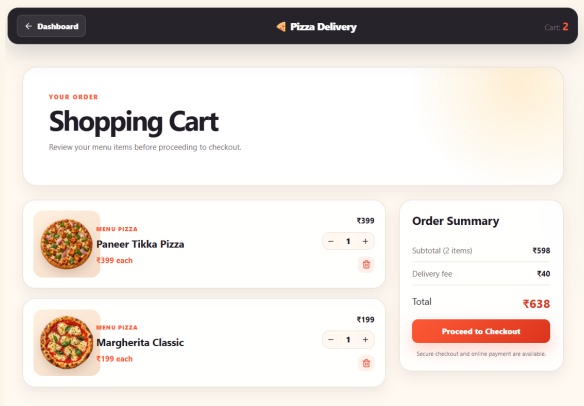


Figure 3. Shopping Cart

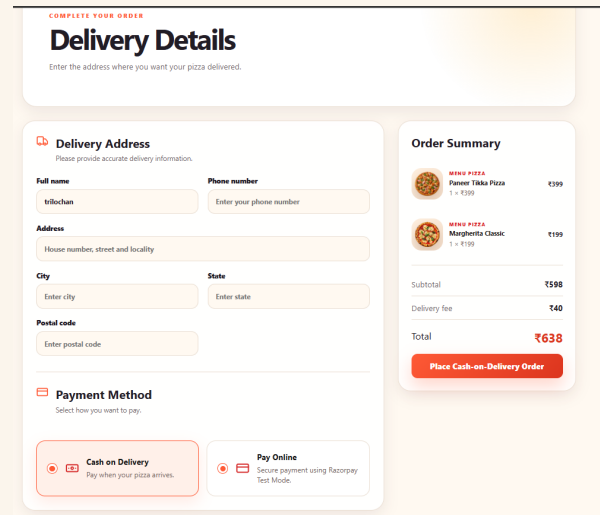


Figure 4. Delivery and payment

Checkout rules

- Quantity changes update the subtotal before checkout.
- A delivery fee is shown separately from the item subtotal.
- Required address fields are validated before an order is submitted.
- Cash on Delivery creates an order with a pending payment state.
- Razorpay uses test mode and verifies the returned payment signature on the backend.

11 / ORDER HISTORY AND TRACKING

A readable record after checkout

My Orders gives customers a durable history instead of ending the experience at a success message. Each order shows its number, date, item snapshots, prices, delivery details and current fulfilment stage.

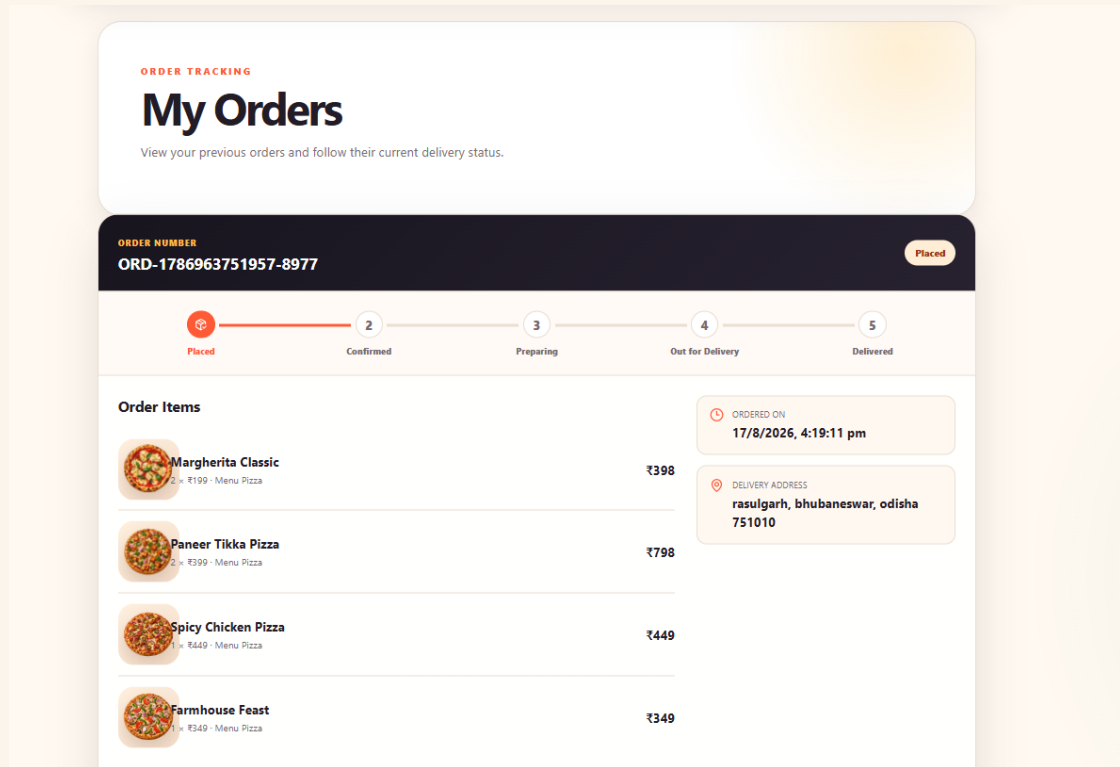


Figure 5. Order history with item details and a five-stage tracking timeline.



Traceable order identity

Every order receives a unique human-readable order number while MongoDB maintains its internal identifier. The public number is easier to communicate during support or fulfilment checks.

12 / ADMINISTRATION PORTAL

Operational visibility in one protected workspace

The admin interface brings together overview metrics, ingredient inventory and customer orders. Its layout mirrors the customer theme while using denser information cards suitable for management work.

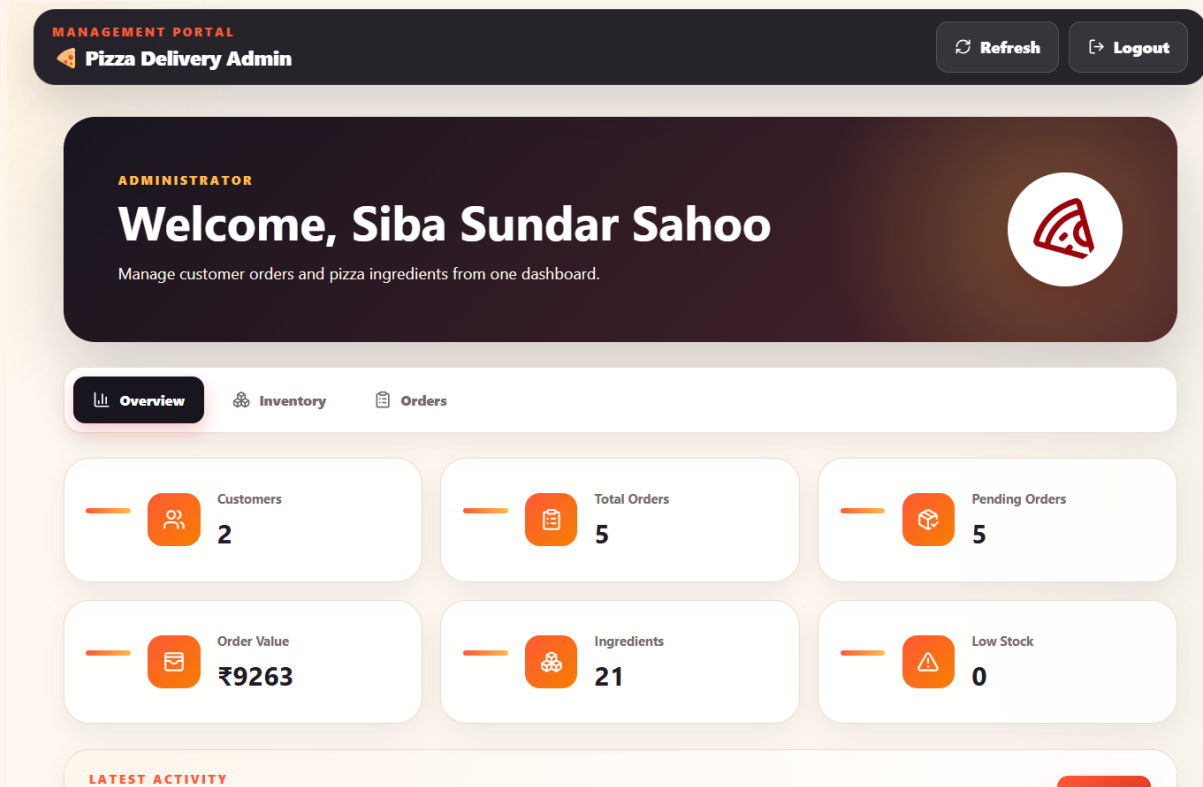


Figure 6. Administrator overview with customers, orders, revenue, inventory and low-stock metrics.

Workspace	Capability
Overview	Summarises customers, total orders, pending orders, order value, ingredient count and low-stock count.
Inventory	Displays ingredient availability and supports stock maintenance.
Orders	Lists customer orders and provides fulfilment-status controls.

13 / API AND PAYMENT INTEGRATION

Backend routes organised by responsibility

The API is divided into functional route modules so that authentication, catalogue, orders, inventory and payment logic can evolve independently. Middleware handles cross-cutting concerns such as token validation, roles, input checks and error responses.

API area	Typical responsibility
Health	Confirms that the Express service and MongoDB connection are available.
Authentication	Registration, verification, login, token refresh, resend verification and password recovery.
Catalogue	Returns menu products and ingredients for browsing and custom pizza construction.
Orders	Creates customer orders, returns order history and updates fulfilment state.
Payments	Creates a Razorpay order and verifies completed payment data.
Administration	Returns business metrics, inventory and operational order controls.

Online payment sequence

- The frontend requests a payment order from the backend.
- The backend calculates or validates the payable amount and uses the Razorpay secret server-side.
- The customer completes the test payment through Razorpay Checkout.
- The frontend returns the payment identifiers and signature to the API.
- The backend verifies the signature before recording the payment as paid.

Important boundary

The Razorpay key ID may be used by the client, but the key secret remains exclusively on the backend. The database and mail credentials follow the same server-only rule.

14 / TESTING AND VALIDATION

Feature checks performed across local and deployed environments

Testing combined direct API checks, browser workflows, role-based verification and production health checks. The final catalogue, customer flow and administrator flow were validated after the interface redesign.

Test area	Validation performed	Result
Authentication	Registration, verification, resend, login and protected-route access	Passed
Catalogue	21 items loaded; category filters, search and product images checked	Passed
Pizza Builder	Base, sauce, cheese and vegetable selection added to Cart	Passed
Cart	Mixed products, quantity changes, removal and total recalculation	Passed
Checkout	Delivery form, COD placement and order creation	Passed
Orders	Item snapshots, totals, status and payment method displayed	Passed
Admin	Overview, Inventory, Orders and status controls	Passed
Responsive UI	Overflow, card grid and circular image behaviour	Passed
Production build	Vite production build completed without errors	Passed
Deployment	Render health success, MongoDB connected, Vercel HTTP 200	Passed

A separate backend health endpoint was used to confirm that the deployed API and MongoDB connection were running. Git commit hashes were also compared with origin/main so the production deployment could be traced to the pushed source state.

15 / DEPLOYMENT AND OPERATIONS

A cloud setup with independent frontend and backend services

The repository is organised as a monorepo, while the deployed application separates the frontend, backend and database. Each service receives only the environment information it requires.

Component	Platform	Production responsibility
Frontend	Vercel	Builds the Vite client and serves the public web application.
Backend	Render	Runs the Express API and connects to external services.
Database	MongoDB Atlas	Stores users, catalogue, ingredients and orders.
Source control	GitHub	Tracks code, documentation and deployment-linked commits.

LIVE APPLICATION

<https://oibsip-pizza-delivery-siba-jade.vercel.app/>

BACKEND HEALTH

<https://oibsip-pizza-delivery-siba.onrender.com/api/health>

SOURCE REPOSITORY

<https://github.com/sibasundarsahoo08-creator/OIBSIP>

Operational note

The Render free service may sleep after inactivity and require a short wake-up period. This behaviour is a hosting-tier constraint, not an application failure. The health endpoint provides a quick way to distinguish a waking service from an API or database problem.

16 / CHALLENGES AND LEARNING

Implementation decisions shaped by real debugging

The strongest learning came from connecting features that initially worked only in isolation. Each issue below required tracing data across the browser, API and database rather than applying a purely visual fix.

Challenge	Resolution	Learning
MongoDB Atlas connection failures	Verified SRV access and network allow-list configuration.	Cloud database errors can originate outside application code.
Email-flow routing	Separated verification, resend and recovery responsibilities across routes and pages.	Authentication UX depends on both backend state and clear frontend navigation.
Custom pizza persistence	Stored custom selections as part of the order-item snapshot.	A flexible UI feature must also have a durable data representation.
Responsive card overflow	Reworked grid constraints, minimum widths and image containers.	A theme must be tested against real catalogue volume, not only one card.
Image consistency	Centralised image resolution and retained product images in order views.	Stable asset mapping is important across Dashboard, Cart, Checkout and Orders.
Deployment verification	Compared Git hashes and tested both the API health response and frontend status.	A successful local build does not by itself prove the deployed version is current.

Personal reflection

The project improved my understanding of how a feature moves through a full-stack system. A button on the frontend is only the visible part; the same feature may require validation, a database model, authentication rules, error handling and deployment configuration. Working through the complete flow made debugging more systematic and helped me separate UI problems from API, database and environment problems.

17 / OUTCOME AND FUTURE SCOPE

A complete foundation with room for operational depth

The final system meets the core internship objective and is usable as a portfolio project. It demonstrates full-stack development, cloud deployment, third-party integration, responsive design and role-based operations within one consistent product.

Delivered outcome

- A publicly accessible customer application and health-checked backend.
- A catalogue of 21 products supported by matching visual assets.
- Custom pizza construction connected to Cart, Checkout and My Orders.
- Email verification and password-recovery account flows.
- COD and Razorpay test-mode payment paths.
- Customer order tracking and an administrator management portal.
- A responsive custom theme and complete repository documentation.

Future enhancements

Enhancement	Expected value
Live rider tracking	Shows real-time delivery position and estimated arrival.
Coupons and loyalty points	Supports promotions and repeat-customer retention.
Reviews and ratings	Adds customer feedback to catalogue decisions.
Push notifications	Provides timely status updates outside the active browser session.
Sales analytics	Adds date-based revenue, product performance and demand trends.
Automated testing	Improves confidence through repeatable API and browser regression checks.

18 / CONCLUSION

From a menu interface to a deployed full-stack product

The Pizza Delivery Platform demonstrates how a common consumer workflow can be translated into a structured software system. The project brings together user identity, product discovery, personalisation, pricing, payment, persistence and administration without treating any one screen as an isolated feature.

The final result is valuable because it works across the full journey. A customer can register, explore the menu, create an order and return later to view it. An administrator can work with the same operational data from a protected interface. The application is deployed, documented and verified, which makes it suitable both as an internship submission and as evidence of practical full-stack development.

Final status

Frontend build successful. Backend and MongoDB health confirmed. Customer workflow tested. Administrator workflow tested. Source pushed to GitHub. Vercel and Render deployments online.

Project references

Resource	Link
Source repository	github.com/sibasundarsahoo08-creator/OIBSIP
Live application	oibsip-pizza-delivery-siba-jade.vercel.app
Backend health	oibsip-pizza-delivery-siba.onrender.com/api/health
React documentation	react.dev
Express documentation	expressjs.com
MongoDB documentation	mongodb.com/docs

Prepared by

Siba Sundar Sahoo

B.Tech - Computer Science and Engineering
ITER, Siksha 'O' Anusandhan
August 2026